

Estrutura Base de um Projeto Spring Boot

Imagina que estás a criar uma aplicação chamada ListaTelefonica.



Explicação dos Principais Diretórios

Diretório/Ficheiro	Função
ListaTelefonicaApplication.java	É o ponto de arranque da aplicação (@SpringBootApplication).
controller/	Contém as classes REST que recebem os pedidos (ex: GET, POST).
service/	Onde vive a lógica de negócio , separada dos controladores.
model/	Define as entidades (ex: Pessoa, Contacto) que são usadas no sistema.
repository/	Interfaces que estendem JpaRepository, para facilitar o acesso à base de dados.
resources/	Onde estão os ficheiros de configuração e templates.
application.properties ou application.yml	Configurações da app (porta, base de dados, etc.).
pom.xml	Ficheiro Maven que define as dependências (Spring Web, JPA, etc.).

Exemplo Prático

Classe Pessoa.java (model):

```
@Entity
public class Pessoa {
    @Id
    @GeneratedValue
    private Long id;
    private String nome;
    private String telefone;
}
```

Interface PessoaRepository.java (repository):

```
public interface PessoaRepository extends JpaRepository<Pessoa, Long> {}
```

Classe PessoaService.java (service):

```
@Service
public class PessoaService {
    @Autowired
    private PessoaRepository repo;

    public List<Pessoa> listarTodas() {
        return repo.findAll();
    }
}
```

Classe PessoaController.java (controller):

```
@RestController
@RequestMapping("/pessoas")
public class PessoaController {
    @Autowired
    private PessoaService service;

    @GetMapping
    public List<Pessoa> listar() {
        return service.listarTodas();
    }
}
```

Dica Cognitiva

Separar bem o projeto em **camadas** (controlador, serviço, repositório, modelo) ajuda-te a:

- **Organizar melhor o código.**
 - **Evitar duplicação de lógica.**
 - **Facilitar testes e manutenção.**
-

🔗 Qual o papel do controller quando usamos React?

O controller continua a ser **essencial** como **ponte entre o frontend e o backend**, mesmo que o frontend esteja noutra projeto (React, Angular, etc.).

🔗 *Ele expõe as **API REST endpoints**:*

Por exemplo:

```
@GetMapping("/pessoas")
public List<Pessoa> listar() {
    return service.listarTodas();
}
```

O React vai chamar este endpoint via `fetch()` ou `axios`, algo como:

```
fetch('http://localhost:8080/pessoas')
  .then(response => response.json())
  .then(data => console.log(data));
```

🏠 Arquitetura típica: Frontend + Backend

React (UI)

↓ (HTTP/JSON via fetch/axios)

Spring Boot

├ controller/	← API REST (ponto de contacto com o React)
├ service/	← Lógica de negócio
├ repository/	← Acesso a base de dados
└ model/	← Entidades (JPA)

? Então e se **não** tivesses o controller?

Sem a camada controller:

- O React **não teria como comunicar com o backend**.
- Estarias a violar o princípio da **separação de responsabilidades** (SRP).
- Perderias a capacidade de aplicar filtros, paginação, validações REST, etc.

🧠 Metáfora Cognitiva:

Imagina o Spring Boot como um **restaurante**:

- O controller é o **atendedor de pedidos** (garçom).

- O service é o **cozinheiro** que prepara a comida.
- O repository é a **despensa com os ingredientes**.
- O React é o **cliente** a fazer pedidos.

Se tirares o garçom (controller)... o cliente fica a gritar na cozinha sem saber a quem se dirigir! 🤖

🧠 Resposta curta:

! Podes omitir a camada service quando o controller apenas faz chamadas diretas e triviais ao repository, sem lógica de negócio adicional.

☑ Situações em que *é aceitável não ter service*:

1. **CRUD extremamente simples:**

Se o controlador apenas faz um `findAll()`, `save()`, `deleteById()`...

Exemplo:

```
@RestController
@RequestMapping("/pessoas")
public class PessoaController {
    @Autowired
    private PessoaRepository repo;
    @GetMapping
    public List<Pessoa> listar() {
        return repo.findAll();
    }
}
```

2. **Projetos protótipo ou MVP:**

Quando estás a fazer algo rápido, com prazo apertado ou ainda a validar ideias.

3. **Aplicações muito pequenas e com baixa complexidade:**

Um microserviço minúsculo que serve apenas para fornecer dados, sem lógica aplicacional.

🚫 Quando **não** deves omitir o service (e é melhor incluí-lo):

1. **Lógica de negócio presente ou futura:**

Se um dia vais precisar de validar dados, chamar APIs externas, fazer transformações, lógica condicional, etc.

👉 Cria o service logo de início. Ajuda a manter o código limpo e modular.

2. Testabilidade:

A camada service é mais fácil de testar **isoladamente** (unit tests), sem ter de simular um controller.

3. Evitar code smells como:

- Fat Controllers (controladores que fazem tudo)
- Repetição de código entre controladores
- Mistura de responsabilidades (violação do princípio SRP)

4. Manutenção a longo prazo:

Separar responsabilidades ajuda quando a aplicação cresce, novas funcionalidades são adicionadas, ou há mais programadores na equipa.

Regra de ouro (pedagógica):

? “Se amanhã precisares de fazer mais do que apenas ‘guardar’ ou ‘listar’... o service devia estar lá hoje.”

Metáfora cognitiva:

Imagina o controller como um **telefone** e o service como uma **pessoa que executa a tarefa**.

Se o telefone ligar diretamente para a despensa (repository), tudo bem...

Mas assim que precisa de pedir *"corta os ingredientes"*, *"verifica alergias"*, *"cozinha por 30 minutos"*, precisas de uma **camada intermediária inteligente**: o service.

Estrutura do Projeto

Backend (Spring Boot)

- **Tecnologias:** Spring Boot, Spring Data JPA, H2 (banco de dados em memória), Maven
- **Pacotes:**
 - **model:** Define a entidade Pessoa
 - **repository:** Interface para acesso aos dados
 - **service:** Contém a lógica de negócio
 - **controller:** Exposição de endpoints REST

Frontend (React)

- **Tecnologias:** React (com Hooks), Axios, React Router
- **Componentes:**
 - **PessoaList:** Lista todas as pessoas
 - **PessoaForm:** Formulário para criar/editar uma pessoa
 - **PessoaDetails:** Exibe detalhes de uma pessoa

Implementação

1. Backend: Spring Boot

a. Configuração Inicial

Utilize o [Spring Initializr](#) com as seguintes dependências:

- Spring Web
- Spring Data JPA
- H2 Database
- Lombok

b. Entidade Pessoa

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Pessoa {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String telefone;
}
```

c. Repositório

```
public interface PessoaRepository extends JpaRepository<Pessoa, Long> {}
```

d. Serviço

```
@Service
public class PessoaService {
    @Autowired
    private PessoaRepository repository;

    public List<Pessoa> listarTodas() {
        return repository.findAll();
    }

    public Pessoa buscarPorId(Long id) {
        return repository.findById(id).orElseThrow(() -> new
RuntimeException("Pessoa não encontrada"));
    }

    public Pessoa salvar(Pessoa pessoa) {
        return repository.save(pessoa);
    }

    public void deletar(Long id) {
        repository.deleteById(id);
    }
}
```

e. Controlador

```
@RestController
@RequestMapping("/api/pessoas")
public class PessoaController {
    @Autowired
    private PessoaService service;

    @GetMapping
    public List<Pessoa> listar() {
        return service.listarTodas();
    }

    @GetMapping("/{id}")
    public Pessoa buscar(@PathVariable Long id) {
        return service.buscarPorId(id);
    }

    @PostMapping
    public Pessoa criar(@RequestBody Pessoa pessoa) {
        return service.salvar(pessoa);
    }
}
```

```

    @PutMapping("/{id}")
    public Pessoa atualizar(@PathVariable Long id, @RequestBody Pessoa
    pessoa) {
        pessoa.setId(id);
        return service.salvar(pessoa);
    }

    @DeleteMapping("/{id}")
    public void deletar(@PathVariable Long id) {
        service.deletar(id);
    }
}

```

f. Configuração do application.properties

```

spring.h2.console.enabled=true
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update

```

2. Frontend: React

a. Configuração Inicial

Crie o projeto React:

```

npx create-react-app frontend
cd frontend
npm install axios react-router-dom

```

b. Serviço de API

```

// src/services/api.js
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8080/api/pessoas',
});

export default api;

```

c. Componente PessoaList

```

// src/components/PessoaList.js
import React, { useEffect, useState } from 'react';
import api from '../services/api';

function PessoaList() {
  const [pessoas, setPessoas] = useState([]);

```



```

useEffect(() => {
  api.get().then((response) => setPessoas(response.data));
}, []);

return (
  <div>
    <h2>Lista de Pessoas</h2>
    <ul>
      {pessoas.map((pessoa) => (
        <li key={pessoa.id}>{pessoa.nome} - {pessoa.telefone}</li>
      ))}
    </ul>
  </div>
);
}

```

```
export default PessoaList;
```

d. Componente PessoaForm

```

// src/components/PessoaForm.js
import React, { useState } from 'react';
import api from '../services/api';

function PessoaForm() {
  const [nome, setNome] = useState('');
  const [telefone, setTelefone] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    api.post('', { nome, telefone }).then(() => {
      setNome('');
      setTelefone('');
    });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Nome"
        value={nome}
        onChange={(e) => setNome(e.target.value)}
      />
      <input
        type="text"
        placeholder="Telefone"
        value={telefone}
        onChange={(e) => setTelefone(e.target.value)}
      />
    </form>
  );
}

```

```

        />
        <button type="submit">Salvar</button>
    </form>
  );
}

export default PessoaForm;

e. Componente Principal App
// src/App.js
import React from 'react';
import PessoaList from './components/PessoaList';
import PessoaForm from './components/PessoaForm';

function App() {
  return (
    <div>
      <h1>Gestão de Pessoas</h1>
      <PessoaForm />
      <PessoaList />
    </div>
  );
}

export default App;

```

Executando o Projeto

1. Backend:

- Navegue até o diretório do projeto Spring Boot.
- Execute: `./mvnw spring-boot:run` (Linux/Mac) ou `mvnw spring-boot:run` (Windows).

2. Frontend:

- Navegue até o diretório frontend.
- Execute: `npm start`.

A aplicação React estará disponível em `http://localhost:3000` e comunicará com o backend em `http://localhost:8080`.

Recursos Adicionais

Para aprofundar seus conhecimentos, recomendo os seguintes tutoriais:

- [Spring Boot + React JS CRUD Example Tutorial](#)
- [Use React and Spring Boot to Build a Simple CRUD App](#)

Esses recursos fornecem exemplos práticos e detalhados que complementam o que foi apresentado aqui.